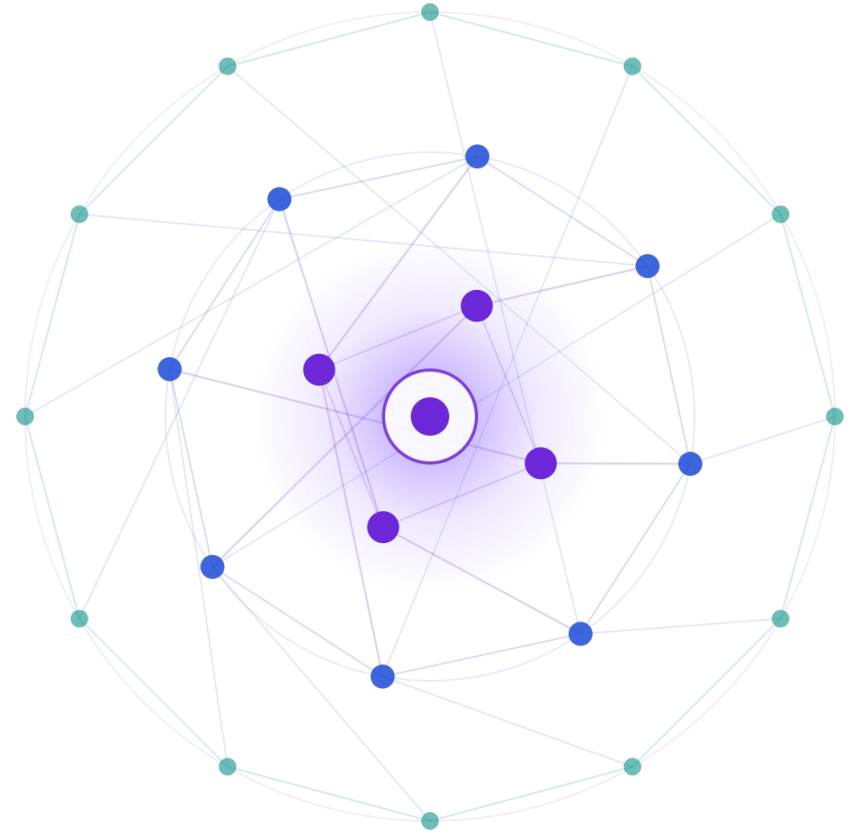


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 1 | Agentic Code Intelligence
- Team Name - Romantic Shers
- College Name - Thapar Institute of Engineering and Technology, Patiala
- Member Name & Email 1- Gurman Singh |
gsingh16_be24@thapar.edu
- Member Name & Email 2- Harkamal Singh |
hlubana_be24@thapar.edu
- Member Name & Email 3- Manraj Singh |
msingh17_be24@thapar.edu
- Member Name & Email 4- Dhruv Srivastava | dhruvsri54@gmail.com
- Submission Github link - github.com/Lagmator22/OvaSearch



Theme

- **Theme 1 | Agentic Code Intelligence.** Given a natural language query, rank code snippets by how relevant they are. Only retrieval is scored, answer generation is out of scope.
- **Why it is hard:** real codebases have thousands of snippets, far past any LLM context window, and asking an LLM to rank all of them is too slow for the first step of a RAG pipeline.
- **Screening:** CoIR AppsRetrieval test split through MTEB. 3765 programming problem statements as queries against 8765 Python solutions, scored with NDCG@10 and MRR.
- **Hands-on round:** P1 | rebuild the index for any version of the code in reasonable time. Bonus | retrieve across versions and handle near-duplicate snippets.
- **Constraint:** everything runs on CPU. We used no GPU at any point, for the benchmark or the demo.
- **Where we landed:** NDCG@10 0.514 | MRR@10 0.470 on CPU, about 9x our bge-small baseline.

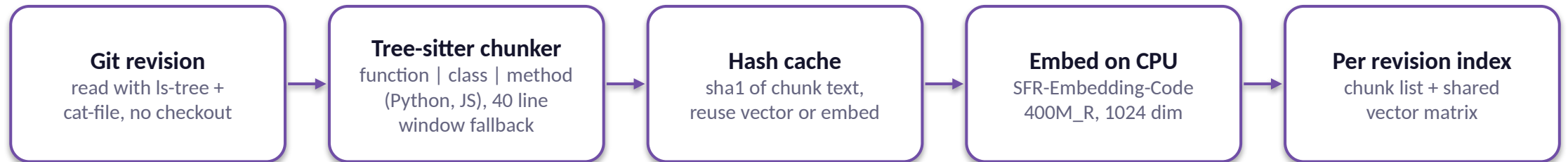
Existing Solutions & Gaps

- **grep / IDE search:** fast and exact, but only works if you already know the identifier name.
- **BM25 keyword search:** on AppsRetrieval BM25 alone scored 0.015 NDCG@10 in our test, because problem statements and Python solutions share almost no words.
- **General text embedders:** semantic but trained on prose. bge-small-en-v1.5 scored only 0.056 NDCG@10.
- **Code embedders differ a lot:** with raw queries we measured jina-embeddings-v2-base-code 0.163, CodeRankEmbed 0.235 and SFR-Embedding-Code-400M_R 0.508. Model choice mattered more than anything else we tried.
- **Hosted assistant codebase search:** works well, but the index lives in the cloud and is closed, so it does not fit a local, CPU only setting.
- **Most tools index one snapshot:** when the code changes they rebuild from scratch and cannot tell you how a function looked in older releases.
- **Gap we target:** local CPU retrieval with a strong code embedder, function level chunks, and an index that is cheap to rebuild per commit and can search across versions.

Our Solutions & Architecture Diagram

prism (our Python package in the OvaSearch repo) has an index path that runs once per git revision and a query path that runs per question. Everything runs on CPU.

INDEX PATH (once per git revision)



QUERY PATH (per question)



--all-versions searches every indexed revision and groups near-duplicates (same symbol, cosine 0.95 or more). Benchmark run uses dense only + query cleanup.

Demo & Product Walkthrough

- **Demo video (max 5 min):** https://drive.google.com/file/d/1lrPoPA8nCpHOSsjEkVldNYA3m0Zxl8St/view?usp=drive_link
- **1. Setup:** Python 3.11 venv, pip install -r requirements.txt (CPU torch), python -m pytest -q runs 17 tests.
- **2. Index three Express.js releases:** python -m prism index /tmp/express --rev 4.18.2 --rev 4.21.2 --rev v5.1.0
- **3. Search:** "set the Content-Type header and send a JSON response" returns res.contentType, res.set and res.json in lib/response.js, in 321 ms.
- **4. P1 rebuild:** 4.21.2 reuses 668 of its 910 chunk vectors from 4.18.2, so the rebuild takes 121 s instead of 318 s cold. Re-indexing a revision already indexed takes 0.19 s.
- **5. Bonus:** --all-versions "redirect to another url with a status code" returns res.redirect once, listing all three releases (similarity 1.000 and 0.988).
- **6. Benchmark:** python -m prism.eval.run_apps_eval --model sfr-400m --query-mode no_examples_compact writes appsretrieval_results.json.

Tools and tech stack used

- **Language:** Python 3.11 for prism (the Theme 1 work). C++17 + CMake for the base OvaSearch engine.
- **Embedding model:** Salesforce SFR-Embedding-Code-400M_R through sentence-transformers on PyTorch CPU. nomic-ai CodeRankEmbed (137M) is supported as a faster, less accurate option.
- **Chunking:** tree-sitter parsers for Python and JavaScript, 40 line windows for everything else.
- **Retrieval:** numpy cosine search over normalised vectors, rank_bm25, reciprocal rank fusion.
- **Versioning:** git ls-tree and cat-file to read any revision, sha1 content hash cache shared across revisions.
- **Evaluation:** MTEB 2.x with our PrePostPipelineEncoder (AbsEncoder subclass) on AppsRetrieval.
- **Testing:** pytest, 17 tests. Setup verified in a fresh venv with an empty Hugging Face cache.
- **Base engine:** Intel OpenVINO + USearch in C++ for document and image RAG. Not yet wired into prism.

Impact & Use case

- **Onboarding on big codebases:** a new developer can ask in plain English and jump straight to the file, line range and symbol.
- **Retrieval layer for coding agents:** an agent pulls the top 10 function level snippets instead of stuffing the whole repo into its context, so fewer tokens and faster answers.
- **Code stays on the machine:** indexing and search run on a laptop CPU, which suits teams that cannot send internal code to a cloud service.
- **Debugging across releases:** search any tag or commit, or all of them at once, to see how a function changed while chasing a regression.
- **Cheap to keep fresh:** a new commit only embeds the chunks that changed, so re-indexing after each merge in CI is realistic.

Innovation highlights, results and limitations

Innovation

- Content hash vector cache shared by all revisions, so only changed chunks are embedded, with no git checkout needed.
- Version families: one result lists every release that has the function, marked identical or near-duplicate.
- Query cleanup for APPS: cut the Examples blocks from problem statements (+0.006 NDCG@10 on two models).

Results (CPU only, Apple M2 Pro)

- AppsRetrieval: NDCG@10 0.514 | MRR@10 0.470 with SFR-Embedding-Code-400M_R + query cleanup, vs 0.056 | 0.048 for bge-small. Full run took 58 min.
- Express.js: cold index 318 s, next release 121 s (668 of 910 chunks reused), re-index 0.19 s. Query latency about 320 to 410 ms.

Limitations

- prism runs on PyTorch CPU. OpenVINO export is not done yet (our CodeRankEmbed trial only matched PyTorch at cosine 0.95 to 0.98).
- BM25 fusion hurt on AppsRetrieval, so the benchmark is dense only. No reranker or query expansion yet.
- AST chunking only for Python and JS. Renamed functions show up as separate families. Brute force search, numbers from one machine.

What's next

- **1. OpenVINO for prism:** export and INT8 quantise the code embedder so queries and indexing get faster on Intel CPUs, then wire prism into the C++ engine.
- **2. Reranking:** a small cross encoder on the top 50 results, plus query expansion for vague questions.
- **3. Code cleanup:** measure the comment and blank line stripping modes we already built (--doc-mode) on AppsRetrieval.
- **4. More languages:** tree-sitter chunking for TypeScript, Java and Go.
- **5. Structural queries:** a call graph from the AST to answer "which files call tool X before tool Y".
- **6. Scale:** an ANN index (USearch HNSW) for very large monorepos, and rename-aware tracking of functions across versions.

Brownie points slide(differentiation)

- **Every choice is measured:** all runs are logged in prism/eval/RESULTS.md, including the ones that failed (BM25 fusion dropped NDCG@10 from 0.235 to 0.228, 0.213 and 0.173).
- **9x the baseline without a GPU:** 0.056 to 0.514 NDCG@10 on a laptop CPU.
- **Real multi-version demo:** three actual Express.js releases, with measured rebuild times and cross-version results, not a toy example.
- **Git native indexing:** reads any tag or commit straight from git objects, so the working tree is never touched.
- **Reproducible:** one requirements.txt, fresh venv tested, 17 tests, and the exact command that produces our submitted JSON.
- **Team experience with OpenVINO:** our team lead is a Google Summer of Code 2026 contributor to Intel's OpenVINO, which is where the CPU inference work goes next.

Checklist- Updated on Public GitHub

- **Working prototype code, public GitHub repo:** Y | github.com/Lagmator22/OvaSearch
- **README with reproducible setup instructions:** Y
- **Demo video, max 5 minutes:**
Y | https://drive.google.com/file/d/1IrPoPA8nCbHOSsjEkVldNYA3m0Zxl8St/view?usp=drive_link
- **Presentation file (PPT or PDF):** Y
- **MTEB AppsRetrieval result JSON in a GitHub release:** Y | github.com/Lagmator22/OvaSearch/releases/tag/PRISM_GENAI_HACKATHON_Y2026

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India